

SYST 17796 DELIVERABLE 2

DESIGN DOCUMENT TEMPLATE

PROJECT: BLACKJACK CARD GAME

TEAM: BINARY BANDITS

DATE SUBMITTED: 2026-07-10

OVERVIEW

1. Project Background and Description

Elaborate on the game you chose and the Description provided in Deliverable 1 by providing more detail on the exact scope of your project (i.e. "the game will terminate after four rounds, giving each player a total score").

Our project extends the Sheridan SYST17796 card game starter code into a playable, console-based Blackjack game. Blackjack is a card game where players compete against the dealer by trying to achieve a hand value as close to 21 as possible without exceeding it. Each player is dealt two cards and may choose to "Hit" to receive another card or "Stand" to keep their current hand. The dealer follows the standard Blackjack rules by drawing cards until reaching at least 17. At the end of each round, the game compares the player's hand with the dealer's hand to determine the winner.

The project builds upon the provided abstract classes (Game, Player, Card, and GroupOfCards) by extending them into Blackjack-specific classes such as BlackjackGame, BlackjackPlayer, Dealer, PlayingCard, Deck, and Hand. Enumerations (Rank and Suit) are used to model the values and suits of playing cards in a type-safe manner. The game continues until the current round is completed, after which the winner is determined and displayed. After each round, the game displays the result and asks whether the players want to play another round. The design emphasizes object-oriented principles such as abstraction, inheritance, encapsulation, loose coupling, and high cohesion to produce a flexible, reusable, and maintainable implementation.

2. Use Case Diagram and Use Case Narratives

Figure 1 displays our use case diagram.

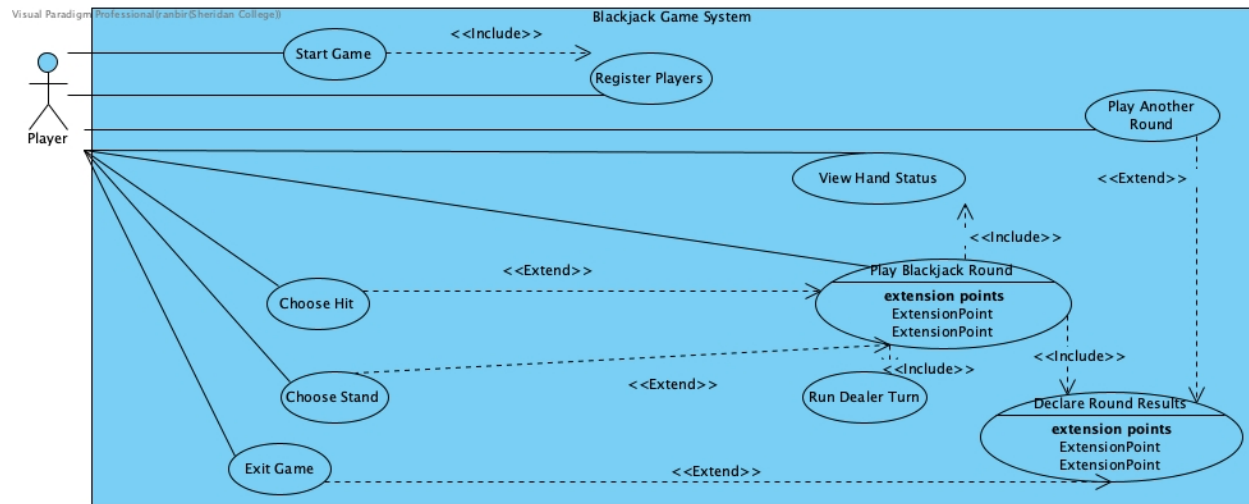


Figure 1: Use Case Diagram

2.1 - Use Case 1: Start Game and Register Players

Item	Description
Use Case Name	Start Game and Register Players
Primary Actor	Player
Goal	Start the Blackjack game and register the players who will participate.
Precondition	The program has been launched from the StartBlackjack class.
Postcondition	The game has created the player objects and is ready to begin the first round.

Main Path

1. The player starts the Blackjack game.
2. The system displays a welcome message and explains the basic Blackjack rules.
3. The system asks how many players will participate.
4. The player enters a number between 1 and 3.
5. The system asks for each player's name.
6. The player enters the names.
7. The system creates the player objects and adds them to the game.
8. The game proceeds to the first Blackjack round.

Alternate Paths

1. **Invalid number of players:** If the player enters a number lower than 1, higher than 3, or a non-numeric value, the system displays an error message and asks again.
 2. **Blank player name:** If the player leaves a name blank, the system automatically assigns a default name such as Player 1.
-

2.2 - Use Case 2: Play Blackjack Round

Item	Description
Use Case Name	Play Blackjack Round
Primary Actor	Player
Goal	Play one complete round of Blackjack against the dealer.
Precondition	At least one player has been registered.
Postcondition	The round is completed and the result is displayed.

Main Path

1. The system creates a new standard 52-card deck.
2. The system shuffles the deck.
3. The system creates a new empty hand for each player and the dealer.
4. The system deals two cards to each player.
5. The system deals two cards to the dealer.
6. The system displays the dealer's first card and hides the second card.
7. Each player takes a turn choosing whether to hit or stand.
8. The dealer takes a turn using the house rule.
9. The system compares each player's total with the dealer's total.
10. The system displays whether each player won, lost, tied, or busted.

Alternate Paths

1. **Player busts:** If a player's hand total goes over 21, the player immediately busts and their turn ends.
2. **Player reaches 21:** If a player reaches exactly 21, the player's turn ends automatically.

3. **Dealer busts:** If the dealer's total goes over 21, all players who did not bust win the round.
-

2.3 - Use Case 3: Choose Hit

Item	Description
Use Case Name	Choose Hit
Primary Actor	Player
Goal	Draw another card to try to improve the player's hand.
Precondition	It is the player's turn and the player's hand total is less than 21.
Postcondition	The player receives one additional card, and the hand total is updated.

Main Path

1. The system displays the player's current hand and total.
2. The system asks the player to choose hit or stand.
3. The player enters "h".
4. The system deals one card from the deck to the player's hand.
5. The system displays the card that was drawn.
6. The system recalculates the player's hand total.
7. The player continues their turn if the total is still under 21.

Alternate Paths

1. **Player busts after hit:** If the new total is greater than 21, the system displays that the player busted and ends the player's turn.
 2. **Player reaches 21 after hit:** If the new total is exactly 21, the system displays that the player has 21 and ends the player's turn.
 3. **Invalid input:** If the player enters anything other than "h" or "s", the system displays an error message and asks again.
-

2.4 - Use Case 4: Choose Stand

Item	Description
Use Case Name	Choose Stand
Primary Actor	Player
Goal	Keep the current hand total and end the player's turn.
Precondition	It is the player's turn and the player has not busted.
Postcondition	The player's turn ends with the current hand total.

Main Path

1. The system displays the player's current hand and total.
2. The system asks the player to choose hit or stand.
3. The player enters "s".
4. The system displays that the player stands at the current total.
5. The system ends the player's turn.

Alternate Paths

1. **Invalid input:** If the player enters anything other than h or s, the system displays a message asking the player to type "h" or "s".
-

2.5 - Use Case 5: Run Dealer Turn

Item	Description
Use Case Name	Run Dealer Turn
Primary Actor	Player
Supporting Class	System-controlled Dealer
Goal	Allow the dealer to complete its turn according to Blackjack house rules.
Precondition	All players have completed their turns.
Postcondition	The dealer has either stood or busted.

Main Path

1. The system reveals the dealer's full hand.
2. The system calculates the dealer's hand total.
3. If the dealer's total is less than 17, the dealer draws another card.
4. The system displays the card drawn by the dealer.
5. The system recalculates the dealer's hand total.
6. The dealer repeats this process until the total is 17 or higher.
7. If the dealer's total is between 17 and 21, the dealer stands.

Alternate Paths

1. **Dealer busts:** If the dealer's total goes over 21, the system displays that the dealer busted.
-

2.6 - Use Case 6: Declare Round Results

Item	Description
Use Case Name	Declare Round Results
Primary Actor	Player
Goal	Determine and display whether each player won, lost, tied, or busted.
Precondition	The player turns and dealer turn have been completed.
Postcondition	The result of the round has been displayed for each player.

Main Path

1. The system gets the dealer's final hand total.
2. The system gets each player's final hand total.
3. The system compares each player's hand total with the dealer's hand total.
4. The system displays the result for each player.
5. The system asks whether the players want to play another round.

Alternate Paths

- **Player busts:** If a player's total is greater than 21, the system declares that player lost.

- **Dealer busts:** If the dealer's total is greater than 21 and the player did not bust, the system declares the player the winner.
 - **Player beats dealer:** If the player's total is greater than the dealer's total and not over 21, the system declares the player the winner.
 - **Player ties dealer:** If the player's total equals the dealer's total, the system declares a push/tie.
 - **Dealer beats player:** If the dealer's total is greater than the player's total and not over 21, the system declares the player lost.
-

2.7 - Use Case 7: Play Another Round or Exit Game

Item	Description
Use Case Name	Play Another Round or Exit Game
Primary Actor	Player
Goal	Decide whether to continue playing or end the game.
Precondition	A Blackjack round has ended.
Postcondition	Either a new round begins or the game ends.

Main Path

1. The system asks whether the players want to play another round.
2. The player enters "y".
3. The system starts a new round with a fresh shuffled deck and new hands.

Alternate Path

1. **Player exits game:** If the player enters anything other than "y", the system ends the game and displays a closing message.
-

3. Class Diagram:

Figure 2 displays our class diagram.

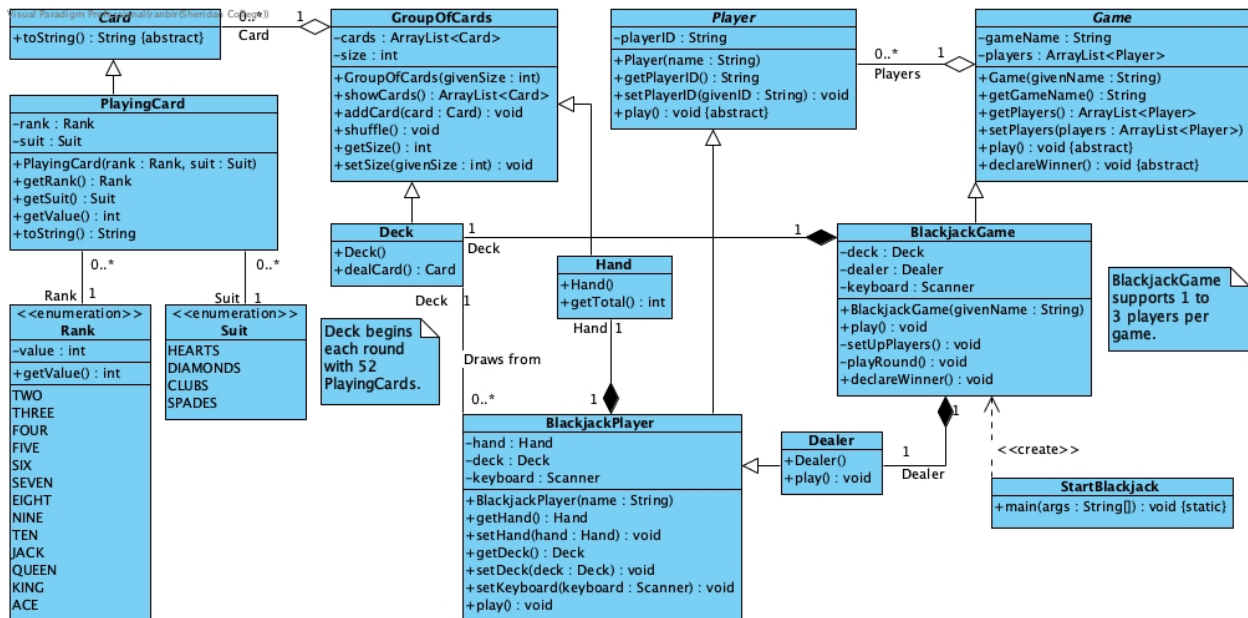


Figure 2: Class Diagram

Figure 2 shows the UML class diagram for our Blackjack game. The diagram extends the starter code by connecting the abstract base classes to Blackjack-specific classes. BlackjackGame inherits from Game, BlackjackPlayer inherits from Player, Dealer inherits from BlackjackPlayer, and PlayingCard inherits from Card. Deck and Hand both inherit from GroupOfCards, which allows them to reuse the card collection and card management methods. The diagram also shows that a Game can have zero or more Player objects, while a GroupOfCards can contain zero or more Card objects. The BlackjackGame class owns one Dealer and one current Deck, and each BlackjackPlayer owns one Hand. The Rank and Suit enumerations are associated with PlayingCard to make card values type-safe.

4. Design Considerations

1. Encapsulation:

Encapsulation is achieved by keeping the internal data of each class private and providing controlled access through public methods. For example, the `PlayingCard` class stores its `Rank` and `Suit` as private final attributes and provides getter methods instead of allowing direct modification. Similarly, `BlackjackPlayer` stores its `Hand`, `Deck`, and `Scanner` as private attributes and provides getter and setter methods only where the rest of the game needs controlled access. This protects the internal state of each object and ensures that modifications occur only through controlled interfaces.

2. Delegation:

Delegation is used throughout the project by assigning responsibilities to the classes best suited to perform them. Rather than having `BlackjackGame` calculate card values or manage hands directly, these responsibilities are delegated to other classes. The `Hand` class calculates Blackjack totals, including special Ace handling, while the `Deck` class creates, stores, shuffles, and deals cards. The `Dealer` class is responsible for implementing the house rules during its turn. This separation of responsibilities simplifies the game logic and makes each class easier to understand and maintain.

3. Cohesion:

The project demonstrates high cohesion because each class focuses on a single responsibility. `PlayingCard` represents one playing card, `Deck` manages the collection of cards available for dealing, `Hand` calculates the total value of cards held by a player, `BlackjackPlayer` manages player decisions, `Dealer` performs dealer-specific behaviour, and `BlackjackGame` coordinates the overall game flow. Since each class performs one well-defined task, future modifications can often be made without affecting unrelated classes.

4. Coupling:

The project is designed with relatively loose coupling. Classes interact through clearly defined public methods rather than accessing each other's internal data directly. For example, `BlackjackGame` requests cards from the `Deck` using `dealCard()` instead of manipulating the deck's internal collection. Likewise, the game obtains a player's score through the `Hand` class using `getTotal()`. The use of the `Rank` and `Suit` enumerations also improves type safety by preventing

invalid card values and reducing dependencies on hard-coded strings or integers.

5. Inheritance:

Inheritance is one of the primary design techniques used in this project. BlackjackGame extends the abstract Game class, BlackjackPlayer extends the abstract Player class, and PlayingCard extends the abstract Card class. In addition, both Deck and Hand inherit from GroupOfCards, allowing them to reuse the collection management functionality already implemented in the starter code. Finally, Dealer inherits from BlackjackPlayer and overrides the play() method to implement the dealer's automatic decision-making behaviour.

6. Aggregation:

Aggregation is shown in the relationships where one class contains a collection of objects, but the contained objects can still be understood as separate objects. In Figure 2, Game has an aggregation relationship with Player, because the game manages a list of players using ArrayList<Player>. The multiplicity shows that one game can have zero or more players. GroupOfCards also has an aggregation relationship with Card, because it stores a collection of cards using ArrayList<Card>. This allows both Deck and Hand to reuse the same card-grouping structure through inheritance. These relationships are “has-a” relationships and help show how the game is organized.

7. Composition:

Composition is shown where one object strongly owns another object. In Figure 2, BlackjackGame has a composition relationship with Dealer, because the dealer is created and controlled by the game. BlackjackGame also owns the current Deck used during a round. BlackjackPlayer has a composition relationship with Hand, because each player has one current hand during the round, and that hand is reset when a new round begins. These relationships are stronger than simple associations because the owned objects are part of the game or player structure.

8. Flexibility/Maintainability:

Our design should be easier to maintain because the major responsibilities are separated into different classes. The use of inheritance allows the starter code to be extended without modifying the original abstract classes. Enumerations (Rank and Suit) improve type safety and simplify future modifications to the card model. Because responsibilities are distributed across specialized classes such as Deck,

Hand, Dealer, and BlackjackPlayer, individual components can be modified or extended with minimal impact on the remainder of the system. This modular structure makes the project easier to debug, test, and enhance in future iterations.

5. References

Ambler, S. W. (n.d.). UML class diagrams: Diagramming guidelines. Agile Modeling. Retrieved July 11, 2026, from <https://agilemodeling.com/style/classDiagram.htm>

Ambler, S. W. (n.d.). UML use case diagrams: Diagramming guidelines. Agile Modeling. Retrieved July 11, 2026, from <https://agilemodeling.com/style/usecasediagram.htm>

Bicycle Cards. (n.d.). Blackjack. Retrieved July 11, 2026, from <https://bicyclecards.com/how-to-play/blackjack>

McLaughlin, B., Pollice, G., & West, D. (2007). Head first object-oriented analysis and design. O'Reilly Media.